

SIMATS ENGINEERING
Co3-ASSESSMENT TOOL 2
Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4

Total Marks:40

Code of Conduct:

I M . h a r i n i (1 9 2 4 2 4 0 1 1) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: m.harini

Criteria	Understanding the case 2.5	Application of Concepts 2.5	Analysis 2	Solution Design 2	Clarity 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

QUESTIONS:4

Total Marks:40

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS, DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3. A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CASE STUDY 1 – CLOUD VIDEO CONFERENCING

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.

The problem is primarily related to the transport protocol and application-level buffering strategy, but for real-time video conferencing, the main concern is usually the choice and configuration of the transport mechanism.

The network has 8% packet loss and 180 ms jitter, which are very high for real-time communication. If TCP is being used for real-time audio and video, its retransmission and congestion-control mechanisms can introduce additional delay. TCP waits for lost packets to be retransmitted, which can cause frozen video and delayed audio.

If UDP is being used, it avoids retransmission delays, but the application must handle packet loss, jitter, and buffering itself. Therefore, an unsuitable application-level buffering strategy can also cause voice distortion and frozen video.

2. Justify the answer based on the communication requirements of real-time multimedia applications.

Real-time multimedia applications require:

- Very low latency
- Low jitter
- Continuous data delivery
- Tolerance to some packet loss
- Adaptive bandwidth usage

For voice and video, receiving data late is often worse than losing a small amount of data. TCP guarantees reliable delivery by retransmitting lost packets, but retransmissions can make old audio/video packets arrive too late to be useful.

UDP is traditionally preferred for real-time multimedia because it has low overhead and does not wait for retransmissions. Modern conferencing systems may also use protocols such as WebRTC with RTP/UDP and congestion-control mechanisms.

The observed 180 ms jitter can cause uneven audio and video playback, while 8% packet loss can cause missing audio and video information. Therefore, both transport behavior and application buffering need to be considered.

3. Recommend suitable protocol-level or application-level improvements.

The following improvements are recommended:

1. Use UDP-based real-time transport where appropriate instead of TCP for media streams.

2. Use adaptive jitter buffers to compensate for variable packet arrival times.
3. Implement adaptive bitrate (ABR) so video quality automatically decreases when bandwidth becomes insufficient.
4. Use packet-loss recovery techniques such as Forward Error Correction (FEC) where appropriate.
5. Use selective retransmission only when the lost data can still arrive before its playback deadline.
6. Apply proper congestion control to prevent network overload.
7. Improve network capacity and prioritize real-time traffic using QoS.
8. Use traffic prioritization for audio/video packets.
9. Deploy geographically distributed media servers to reduce network distance.
10. Monitor packet loss, latency, jitter, and bandwidth continuously.

Conclusion

The issue is not caused by only TCP/UDP or only the application layer. The combination of high packet loss, excessive jitter, transport behavior, and buffering strategy is responsible. For real-time conferencing, low-latency transport, adaptive buffering, adaptive bitrate, and QoS can significantly improve quality.

CASE STUDY 2 – GLOBAL DNS INFRASTRUCTURE

4. Analyze the reasons behind the high DNS resolution time.

The average DNS lookup time of 800 ms is extremely high and can significantly delay application startup.

Possible reasons include:

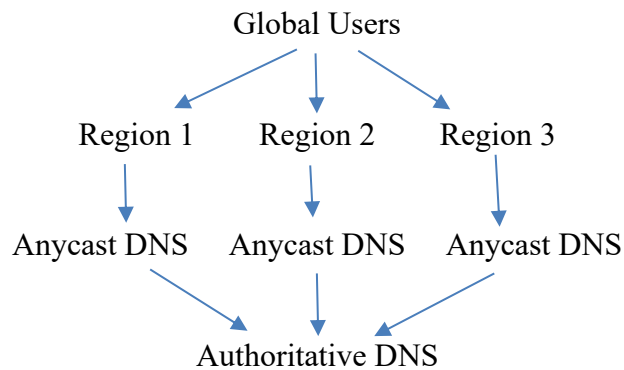
- DNS server is geographically far from users.
- All DNS requests may be handled by a small number of servers.
- High DNS server workload.
- Network congestion.
- Slow communication between recursive resolvers and authoritative DNS servers.
- DNS cache misses.
- Excessive DNS query chains.
- Poorly configured TTL values.
- DNS server failures or retries.
- Lack of geographically distributed DNS infrastructure.

If the DNS server is located far away from the user, the query may require multiple network round trips, increasing resolution time.

5. Propose a suitable DNS architecture to reduce lookup time below 50 ms.

A distributed Anycast DNS architecture should be implemented.

A suitable architecture is:



Anycast DNS

With Anycast, multiple DNS servers use the same IP address and advertise it from different geographical locations.

A user's DNS request is normally routed to a nearby available DNS server.

For example:

User in India → Indian DNS node

User in USA → US DNS node

User in Europe → European DNS node

This reduces network distance and lookup latency.

DNS Pre-fetching

Frequently accessed DNS records can be refreshed before their TTL expires. This helps prevent cache expiration from causing repeated lookup delays.

TTL Optimization

TTL should be selected carefully.

- Very low TTL → more DNS queries and higher DNS-server load.
- Very high TTL → better caching but slower propagation of changes.

A moderate TTL should be selected according to the application's requirements.

Additional improvements

- Use multiple authoritative DNS servers.
- Deploy DNS servers in multiple geographical regions.
- Use DNS caching.
- Monitor DNS response time.
- Remove unnecessary DNS lookup chains.
- Use fast and reliable recursive resolvers.
- Configure health checks and automatic failover.

With nearby Anycast nodes and effective caching, DNS lookup latency can potentially be reduced substantially, although achieving below 50 ms globally cannot be guaranteed for every user or every uncached lookup.

6. Evaluate the advantages and limitations.

Technique	Advantages	Limitations
Anycast DNS	Low latency, high availability, geographic distribution	More complex to deploy and manage
DNS caching	Reduces repeated queries and lookup time	Cached records may become outdated
DNS pre-fetching	Reduces cache-miss delays	Generates additional DNS traffic
TTL optimization	Balances caching and update speed	Incorrect TTL can increase load or delay updates
Multiple DNS servers	Provides fault tolerance	Requires synchronization and monitoring

Performance

Anycast and caching reduce lookup latency by directing users to nearby DNS infrastructure and avoiding unnecessary repeated queries.

Scalability

Distributed DNS servers can handle large numbers of requests because traffic is spread across multiple locations.

Availability

If one DNS server or region fails, traffic can be routed to another available DNS node.

Conclusion

A distributed Anycast DNS architecture combined with caching, DNS pre-fetching, optimized TTL values, and multiple authoritative servers provides better **performance, scalability, and availability** than a centralized DNS system.

CASE STUDY 3 – CENTRALIZED DNS ARCHITECTURE

7. Examine the impact of centralized DNS architecture on application performance.

In the given situation, all DNS requests are directed to a single primary DNS server.

This creates a major bottleneck.

During promotional events, millions of users may generate DNS requests simultaneously. The single DNS server can become overloaded, resulting in:

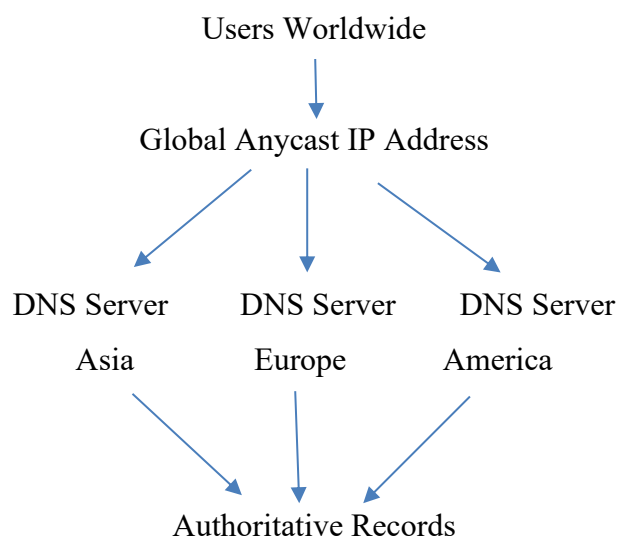
- Increased DNS lookup latency
- Request queuing
- Higher CPU and memory utilization
- Packet loss
- DNS timeouts
- Repeated DNS queries
- Poor application response time

An 800 ms DNS lookup means users may wait nearly one second before the application can begin connecting to the required service.

The centralized server also represents a single point of failure. If it becomes unavailable, users may be unable to resolve the application's domain.

8. Design a distributed DNS solution.

A suitable solution is a globally distributed Anycast DNS architecture.



Working

1. Deploy DNS servers in multiple geographical regions.
2. Configure the servers with the same Anycast IP address.
3. Use BGP Anycast routing to advertise the DNS service.
4. Users are routed toward a nearby DNS node.
5. Use DNS caching to reduce repeated queries.
6. Use health checks to detect failed DNS nodes.
7. Withdraw routes from unhealthy DNS servers.
8. Automatically direct traffic to healthy nodes.

9. Analyze TTL, Anycast routing and DNS caching.

TTL

TTL determines how long a DNS record can remain in a cache.

Higher TTL:

- More caching
- Fewer DNS queries
- Lower DNS load
- Better performance

But changes take longer to propagate.

Lower TTL:

- Faster record updates
- More DNS queries
- Higher DNS server load

Therefore, TTL should be optimized according to the application's needs.

Anycast Routing

Anycast allows multiple DNS servers to advertise the same IP address.

Advantages:

- Reduces geographical latency
- Distributes traffic
- Improves availability
- Provides automatic routing around failures

If one DNS location fails, routing can direct users to another healthy location.

DNS Caching

Caching stores DNS responses closer to users.

For example:

First request → DNS server → Response

Second request → Local cache → Response immediately

Caching greatly reduces lookup latency and decreases the load on authoritative DNS servers.

Conclusion

The centralized DNS design creates a performance bottleneck and single point of failure. A distributed architecture using Anycast routing, DNS caching, optimized TTL values, and multiple DNS servers provides lower latency, better scalability, and higher fault tolerance.

CASE STUDY 4 – HIGH-FREQUENCY STOCK TRADING

10. Analyze three TCP-related factors responsible for the latency increase.

The increase in order acknowledgment latency from 1 ms to 40 ms can be associated with several TCP mechanisms. Three important factors are:

1. Flow control
2. Nagle's algorithm
3. SYN queue limitations

1. TCP Flow Control

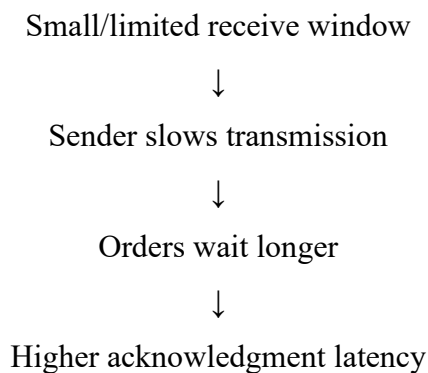
TCP uses a receive window to prevent the sender from transmitting more data than the receiver can handle.

If the receiver's buffer becomes full, the advertised receive window decreases.

This can cause the sender to slow down or temporarily stop transmitting.

In a high-frequency trading environment, excessive buffering or a small receive window can increase order processing latency.

Effect



2. Nagle's Algorithm

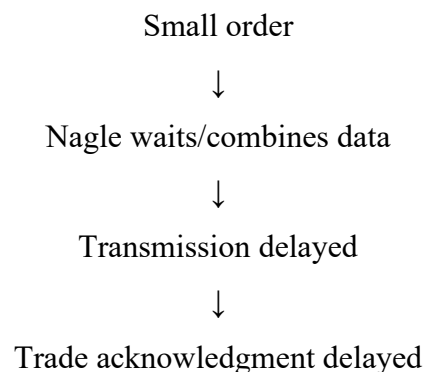
Nagle's algorithm combines small TCP segments into larger segments to reduce network overhead.

This is useful for normal applications but can be harmful to latency-sensitive trading systems.

If a trading application sends a small order message, Nagle's algorithm may delay sending it while waiting for an acknowledgment or additional data.

This can increase transaction latency.

Effect



For latency-sensitive applications, Nagle's algorithm is commonly disabled using: `TCP_NODELAY`

3. SYN Queue Limitations

When a client establishes a TCP connection, the server must process the TCP three-way handshake:

`SYN → SYN-ACK → ACK`

During market opening, thousands of clients may attempt connections simultaneously.

If the server's SYN backlog becomes full, new connection requests may be delayed or dropped.

This can cause:

- Longer connection establishment time
- Retransmitted SYN packets
- Connection delays
- Increased latency

For high-frequency systems, repeatedly creating new TCP connections can therefore be expensive.

11. Explain how each factor affects transaction processing.

Factor	Effect on Trading System
Flow control	Can slow transmission when receive buffers are congested
Nagle's algorithm	Can delay small order messages
SYN queue limitation	Can delay new TCP connections during traffic spikes
TCP retransmissions	Cause additional delay when packets are lost
Excessive buffering	Increases queueing and processing delay

High-frequency trading requires extremely low and predictable latency. Even small delays can affect order submission and acknowledgment.

12. Recommend TCP configuration changes.

The following changes can reduce latency:

1. Disable Nagle's Algorithm

Use:

`TCP_NODELAY`

This allows small trading messages to be transmitted immediately rather than being unnecessarily combined.

2. Optimize TCP Receive and Send Buffers

Configure appropriate TCP buffer sizes based on traffic patterns.

Buffers should be large enough to prevent unnecessary blocking but should not be excessively large because excessive buffering can increase latency.

3. Increase SYN Backlog

Increase the server's TCP SYN backlog so that it can handle bursts of incoming connection requests. This helps reduce connection establishment delays during market opening.

4. Reduce Packet Loss

Investigate network congestion, faulty links, overloaded interfaces, and routing problems. Reducing packet loss reduces TCP retransmissions.

5. Use Persistent TCP Connections

Instead of creating a new TCP connection for every order, maintain established connections where appropriate.

This avoids repeatedly performing:

SYN → SYN-ACK → ACK

and reduces connection establishment latency.

6. Apply Quality of Service

Prioritize trading traffic over less important network traffic.

7. Monitor TCP Retransmissions

Use network monitoring and packet capture tools such as Wireshark to identify retransmissions and congestion.

8. Optimize the Application

Reduce unnecessary buffering and ensure that the application sends critical trading messages immediately.

Conclusion

The increase in trading latency can be caused by **TCP flow control, Nagle's algorithm, SYN queue limitations, retransmissions, and excessive buffering**. Disabling Nagle's algorithm, optimizing TCP buffers, increasing the SYN backlog, reducing packet loss, and maintaining persistent connections can significantly reduce latency while maintaining reliable TCP communication.